Taller: Triggers BEFORE y AFTER (Automatización)

Base de Datos II · Semana 7 · Sesión 1 · CORHUILA 2026

---

1. Descripción del escenario

Campo	Detalle
Objetivo	Automatizar reglas dentro de PostgreSQL usando triggers BEFORE y AFTER
Motor	PostgreSQL
Tablas base	`estudiante`, `matricula`
Tabla de auditoría	`matricula_audit`
Triggers creados	4 triggers (2 BEFORE en `estudiante`, 1 BEFORE en `matricula`, 1 AFTER en `matricula`)

---

2. Conceptos clave

Concepto	Descripción
Trigger	Mecanismo que ejecuta una función automáticamente ante un evento (`INSERT`, `UPDATE`, `DELETE`)
Trigger function	Función especial con `RETURNS trigger` que contiene la lógica a ejecutar
`NEW`	Fila nueva disponible en `INSERT` y `UPDATE`
`OLD`	Fila anterior disponible en `UPDATE` y `DELETE`
`FOR EACH ROW`	El trigger se ejecuta una vez por cada fila afectada (más común)
`FOR EACH STATEMENT`	El trigger se ejecuta una vez por sentencia completa
`TG_OP`	Variable especial que indica la operación actual: `'INSERT'`, `'UPDATE'` o `'DELETE'`

Disponibilidad de NEW y OLD por evento

Evento	`NEW` disponible	`OLD` disponible	Uso típico
`INSERT`	✅	❌	Asignar `created_at`, normalizar campos
`UPDATE`	✅	✅	Comparar cambios, actualizar `updated_at`
`DELETE`	❌	✅	Registrar qué fila fue eliminada

---

3. Tablas base del taller

Tabla	Columnas principales	Propósito
`estudiante`	`id`, `nombre`, `created_at`, `updated_at`	Almacena estudiantes; prueba timestamps y normalización
`matricula`	`id`, `estudiante_id`, `nota_final`, `created_at`, `updated_at`	Almacena matrículas; prueba validación de nota
`matricula_audit`	`id`, `operacion`, `matricula_id`, `fecha`, `detalle`	Registra automáticamente todos los cambios en `matricula`

---

4. Triggers BEFORE creados

4.1. Resumen de triggers BEFORE

Trigger	Tabla	Evento(s)	Función asociada
`trg_estudiante_timestamps`	`estudiante`	`INSERT`, `UPDATE`	`fn_set_timestamps()`

Trigger	Tabla	Evento(s)	Función asociada
`trg_estudiante_normalizar_nombre`	`estudiante`	`INSERT`, `UPDATE`	`fn_normalizar_nombre_est`
`trg_matricula_timestamps`	`matricula`	`INSERT`, `UPDATE`	`fn_set_timestamps()`
`trg_matricula_validar_nota`	`matricula`	`INSERT`, `UPDATE`	`fn_validar_nota()`

4.2. Lógica de fn_set_timestamps()

Operación (`TG_OP`)	Acción sobre `NEW`	Retorno
`INSERT`	Asigna `NEW.created_at = NOW()` si es `NULL`; siempre asigna `NEW.updated_at = NOW()`	`RETURN NEW`
`UPDATE`	Solo actualiza `NEW.updated_at = NOW()` (no toca `created_at`)	`RETURN NEW`

4.3. Lógica de fn_normalizar_nombre_estudiante()

Entrada (`NEW.nombre`)	Transformación aplicada	Resultado
`'  juan perez  '`	`INITCAP(TRIM(...))`	`'Juan Perez'`
`'MARIA LOPEZ'`	`INITCAP(TRIM(...))`	`'Maria Lopez'`
`'  carlos  '`	`INITCAP(TRIM(...))`	`'Carlos'`

4.4. Lógica de fn_validar_nota()

Condición evaluada	Acción	Retorno
`nota_final` es `NULL`	No hace nada, deja pasar	`RETURN NEW`
`nota_final` entre 0 y 5	Válido, deja pasar	`RETURN NEW`
`nota_final < 0` o `nota_final > 5`	Lanza `RAISE EXCEPTION` con el valor recibido	Aborta la operación ❌

5. Trigger AFTER creado

5.1. Resumen del trigger AFTER

Trigger	Tabla	Evento(s)	Función asociada	Propósito
`trg_matricula_audit`	`matricula`	`INSERT`, `UPDATE`, `DELETE`	`fn_audit_matricula()`	Registra cada cambi en `matricula_audit` con operación, ID y fecha

5.2. Lógica de fn_audit_matricula()

Operación (`TG_OP`)	Dato registrado en `matricula_audit`	Retorno
`INSERT`	`operacion='INSERT'`, `matricula_id=NEW.id`, `detalle='Se insertó una matrícula'`	`RETURN NEW`
`UPDATE`	`operacion='UPDATE'`, `matricula_id=NEW.id`, `detalle='Se actualizó una matrícula'`	`RETURN NEW`
`DELETE`	`operacion='DELETE'`, `matricula_id=OLD.id`, `detalle='Se eliminó una matrícula'`	`RETURN OLD`

> **Nota:** En triggers `AFTER` el retorno **no modifica** la fila (ya fue guardada), pero se debe retornar `NEW` u `OLD` según el evento para que PostgreSQL no genere advertencias.

6. Diferencia clave: BEFORE vs AFTER

Característica	BEFORE	AFTER
¿Cuándo se ejecuta?	Antes de guardar la fila	Después de guardar la fila
¿Puede modificar NEW?	✅ Sí	❌ No (ya fue guardada)
¿Puede cancelar la operación?	✅ Sí (con `RAISE EXCEPTION` o retornando `NULL`)	❌ No directamente
Uso principal	Normalización, autocompletar campos, validaciones	Auditoría, logs, notificaciones, acciones en otras tablas
Retorno típico	`RETURN NEW`	`RETURN NEW` o `RETURN OLD` según el evento

---

7. Plan de pruebas

#	Prueba	Sentencia ejecutada	Resultado esperado
1	Timestamps automáticos en INSERT	`INSERT INTO estudiante(nombre) VALUES ('ana garcia');`	`created_at` y `updated_at` se asignan automáticamente ✅
2	Normalización del nombre	`INSERT INTO estudiante(nombre) VALUES ('  juan PEREZ  ');`	`nombre` se guarda como `'Juan Perez'` ✅
3	Timestamps en UPDATE	`UPDATE estudiante SET nombre = 'Ana García López' WHERE ...;`	`updated_at` cambia; `created_at` permanece igual ✅
4	INSERT válido en matrícula (nota = 3.8)	`INSERT INTO matricula(estudiante_id, nota_final) VALUES (1, 3.8);`	Insertado OK + registro en `matricula_audit` ✅
5	INSERT con nota inválida (nota = 6)	`INSERT INTO matricula(estudiante_id, nota_final) VALUES (1, 6.0);`	ERROR: `Nota fuera de rango (0 a 5). Valor = 6` ❌

#	Prueba	Sentencia ejecutada	Resultado esperado
6	UPDATE de matrícula	`UPDATE matricula SET nota_final = 4.2 WHERE id = 1;`	`matricula_audit` registra operación `'UPDATE'` ✅
7	DELETE de matrícula	`DELETE FROM matricula WHERE id = 1;`	`matricula_audit` registra operación `'DELETE'` ✅

8. Resumen de funciones y triggers del taller

Función / Trigger	Tipo	Tabla	Evento	Descripción
`fn_set_timestamps()`	Función trigger	Reutilizable	`INSERT`, `UPDATE`	Gestiona `created_at` y `updated_at` automáticamente
`trg_estudiante_timestamps`	Trigger BEFORE	`estudiante`	`INSERT`, `UPDATE`	Aplica `fn_set_timestar` sobre estudiante
`trg_matricula_timestamps`	Trigger BEFORE	`matricula`	`INSERT`, `UPDATE`	Aplica `fn_set_timestar` sobre matrícula
`fn_normalizar_nombre_estudiante()`	Función trigger	`estudiante`	`INSERT`, `UPDATE`	Limpia y capitaliza nombre
`trg_estudiante_normalizar_nombre`	Trigger BEFORE	`estudiante`	`INSERT`, `UPDATE`	Aplica normalizació de nombre
`fn_validar_nota()`	Función trigger	`matricula`	`INSERT`, `UPDATE`	Valida que `nota_final` esté entre 0 y 5
`trg_matricula_validar_nota`	Trigger BEFORE	`matricula`	`INSERT`, `UPDATE`	Aplica validación de nota
`fn_audit_matricula()`	Función trigger	`matricula`	`INSERT`, `UPDATE`, `DELETE`	Inserta registro en `matricula_audi`

Función / Trigger	Tipo	Tabla	Evento	Descripción
`trg_matricula_audit`	Trigger AFTER	`matricula`	`INSERT`, `UPDATE`, `DELETE`	Aplica auditoría de matrícula

9. Resumen de ideas clave (cierre)

Idea	Descripción
Trigger = automatización	Se ejecuta automáticamente ante `INSERT`, `UPDATE` o `DELETE`, sin intervención manual
BEFORE → modificar o rechazar	Ideal para normalización de texto, autocompletar campos y validaciones que deben abortar la operación
AFTER → registrar o reaccionar	Ideal para auditoría y logs: la operación ya ocurrió, solo se reacciona a ella
NEW / OLD dependen del evento	`NEW` trae los datos nuevos; `OLD` trae los datos anteriores; en `DELETE` solo existe `OLD`
Reutilización de funciones	Una misma función trigger (`fn_set_timestamps`) puede usarse en múltiples tablas

Material elaborado para: Base de Datos II · Unidad 2 · Semana 7 · Sesión 1 · CORHUILA 2026